COMPETITION BRIEF | MARCH 28 ~ APRIL 8, 2026

Scaler × Meta PyTorch Hackathon

Round 1: Building an OpenEnv Environment

Build a complete, real-world OpenEnv environment that an AI agent can learn from through the standard `step()` / `reset()` / `state()` API.

1 Core Objective

Build a **complete OpenEnv environment** simulating a genuine real-world task (not games or toys). The environment must expose the standard OpenEnv interface and be deployable as a containerized Hugging Face Space.

Valid Domain Examples

Email triage, code review, data cleaning, scheduling, customer support, content moderation, financial analysis, inventory management.

2 Functional Requirements

2.1 OpenEnv Spec Compliance

Full implementation of the OpenEnv interface:

- Typed `Observation`, `Action`, and `Reward` Pydantic models
- `step(action)` → returns observation, reward, done, info
- `reset()` → returns initial observation
- `state()` → returns current state
- `openenv.yaml` with metadata
- Must pass `openenv validate`

2.2 Tasks and Graders (Minimum 3)

Each task defines a concrete objective with a programmatic grader scoring performance on a **0.0 to 1.0** scale. Required difficulty progression:

1. **Easy:** Straightforward task, baseline agent should score well
2. **Medium:** Requires multi-step reasoning or planning
3. **Hard:** Should genuinely challenge frontier models

Graders must have **clear, deterministic** success/failure criteria.

2.3 Reward Function

- Signal over the **full trajectory** (not just binary end-of-episode)
- Rewards **partial progress** toward task completion
- Penalizes clearly undesirable behavior (infinite loops, destructive actions)

2.4 Baseline Inference Script

- File: `inference.py` in the project root
- Uses the **OpenAI API client** for all LLM calls
- Reads credentials from environment variables
- Produces reproducible baseline scores on all 3+ tasks

Required Environment Variables

`API_BASE_URL`	LLM API endpoint
`MODEL_NAME`	Model identifier
`HF_TOKEN`	Hugging Face / API key
`OPENAI_API_KEY`	OpenAI API key

3 Non-Functional Requirements

3.1 Deployment

- Deploy to a **Hugging Face Space** tagged with `openenv`
- Must include a working `Dockerfile`
- Clean start via `docker build` + `docker run`

3.2 Infrastructure Constraints

- Inference script runtime: $<$ **20 minutes**
- Must run on: **vCPU=2, memory=8GB**

3.3 Documentation (README)

Must include: environment description and motivation, action/observation space definitions, task descriptions with expected difficulty, setup and usage instructions, baseline scores.

4 Scoring Breakdown

Parameter	Weight	Focus
Real-world utility	30%	Genuine task? Would someone use this to train/evaluate agents?
Task & grader quality	25%	Well-defined tasks, accurate graders, meaningful difficulty progression
Environment design	20%	Clean state, sensible spaces, good reward shaping, proper episode boundaries
Code quality & spec	15%	OpenEnv spec, clean structure, typed models, tested, Dockerfile works
Creativity & novelty	10%	Novel domain, interesting mechanics, clever reward design

5 Detailed Scoring Rubric

5.1 Real-World Utility (30%)

Score	Description
0 – 5	Toy/artificial problem, no practical application
6 – 15	Valid domain but shallow modeling
16 – 25	Good domain modeling, useful for agent evaluation
26 – 30	Excellent; fills a real gap, immediate value for the RL/agent community

5.2 Task & Grader Quality (25%)

- 3+ tasks with difficulty range?
- Graders produce scores in [0.0, 1.0]?
- Graders deterministic and reproducible?
- Hard task genuinely challenges frontier models?

5.3 Environment Design (20%)

- `reset()` produces clean state?
- Action/observation types well-designed and documented?
- Reward function provides useful varying signal (not sparse)?
- Episode boundaries sensible?

5.4 Code Quality & Spec (15%)

- `openenv validate` passes?
- `docker build && docker run` works?
- HF Space deploys and responds?
- Baseline script runs and reproduces scores?

5.5 Creativity & Novelty (10%)

- Domain not yet seen in OpenEnv?
- Reward design with interesting properties?
- Clever mechanics that make the environment engaging?

6 Evaluation Pipeline

6.1 Phase 1: Automated Validation (Pass/Fail Gate)

1. HF Space deploys (ping URL, must return 200 and respond to `reset()`)
2. OpenEnv spec compliance (`openenv.yaml`, typed models, endpoints)
3. `Dockerfile` builds successfully
4. Baseline script completes without error and produces scores
5. 3+ tasks with graders, all scores in [0.0, 1.0]

Failing any of these is an automatic disqualification.

6.2 Phase 2: Agentic Evaluation (Scored)

- Baseline agent re-run
- Standard Open LLM agent (e.g., Nemotron 3 Super) run against all environments
- Score variance check

6.3 Phase 3: Human Review

Top submissions reviewed by **Meta and Hugging Face engineers** for real-world utility, creativity, and exploit checks.

7 Disqualification Criteria

1. Environment does not deploy or respond
2. Plagiarized or trivially modified existing environments
3. Graders that always return the same score
4. No baseline inference script

8 Key API Contract

OpenEnv Interface Summary

```
Models (Pydantic):
  Observation, Action, Reward

Methods:
  step(action: Action)
    → (Observation, Reward, bool, dict)
  reset() → Observation
  state() → State

Config:
  openenv.yaml (metadata + task defs)

Validation:
  openenv validate
```

9 Implementation Notes

For knowledge transfer to subsequent implementation chats:

- The winning strategy optimizes for **real-world utility** (30%) and **task/grader quality** (25%) first; together they account for over half the total score.
- The hard task must *genuinely* challenge frontier models. Design for this explicitly.
- Reward shaping is critical: sparse binary rewards will score poorly on environment design (20%).
- The inference script must use the **OpenAI client library**, not raw HTTP or other SDKs.
- Infrastructure ceiling is tight: 2 vCPUs, 8GB RAM, 20-minute inference cap. Design accordingly.
- The `openenv validate` command is the first automated gate. Run it locally before submission.
- HF Space must respond to a raw `reset()` ping at the Space URL. Test this.

Deadline

April 8, 2026 11:59 PM
Only team leaders can make the final submission.